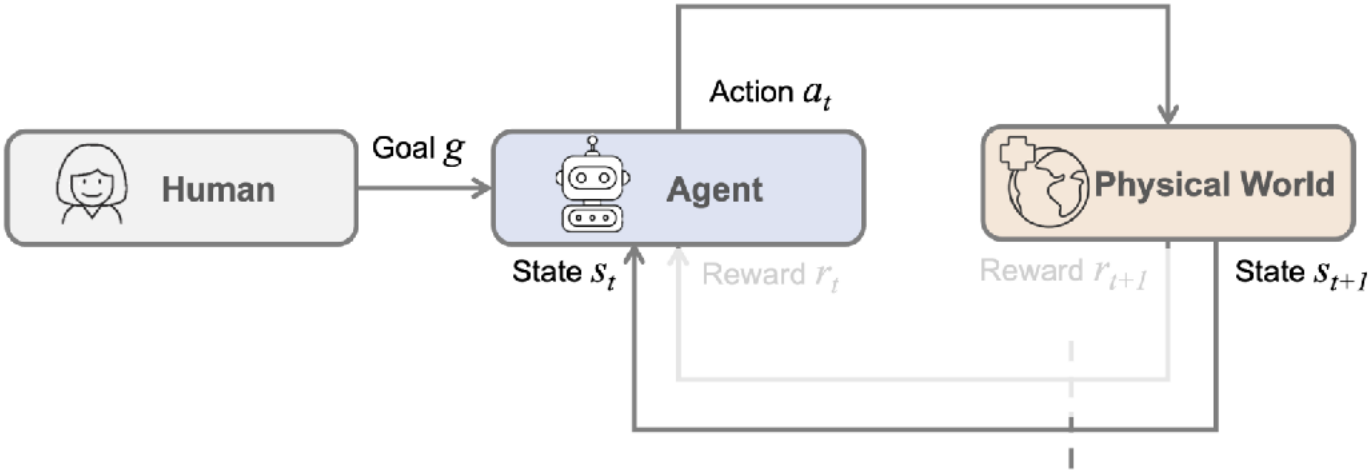
Today’s Outline

- Vision Language Models for Robotic System

Robotic Foundation Models

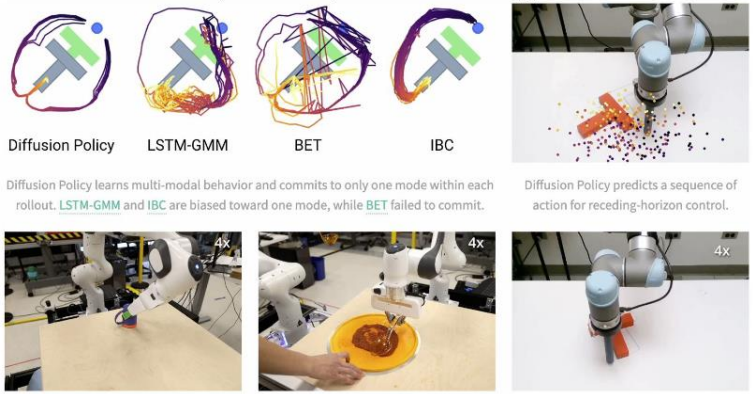
- What is a Robotic Foundation Model?
  - No explicit representation of states / transition functions
  - A policy that maps (observation/state, goal) to action



Robotic Foundation Models

- What is a Robotic Foundation Model?
  - No explicit representation of states / transition functions
  - A policy that maps (observation/state, goal) to action

Imitation Learning
(Chi et al., Diffusion Policy)



Reinforcement Learning
(OpenAI, Solving Rubik's Cube)

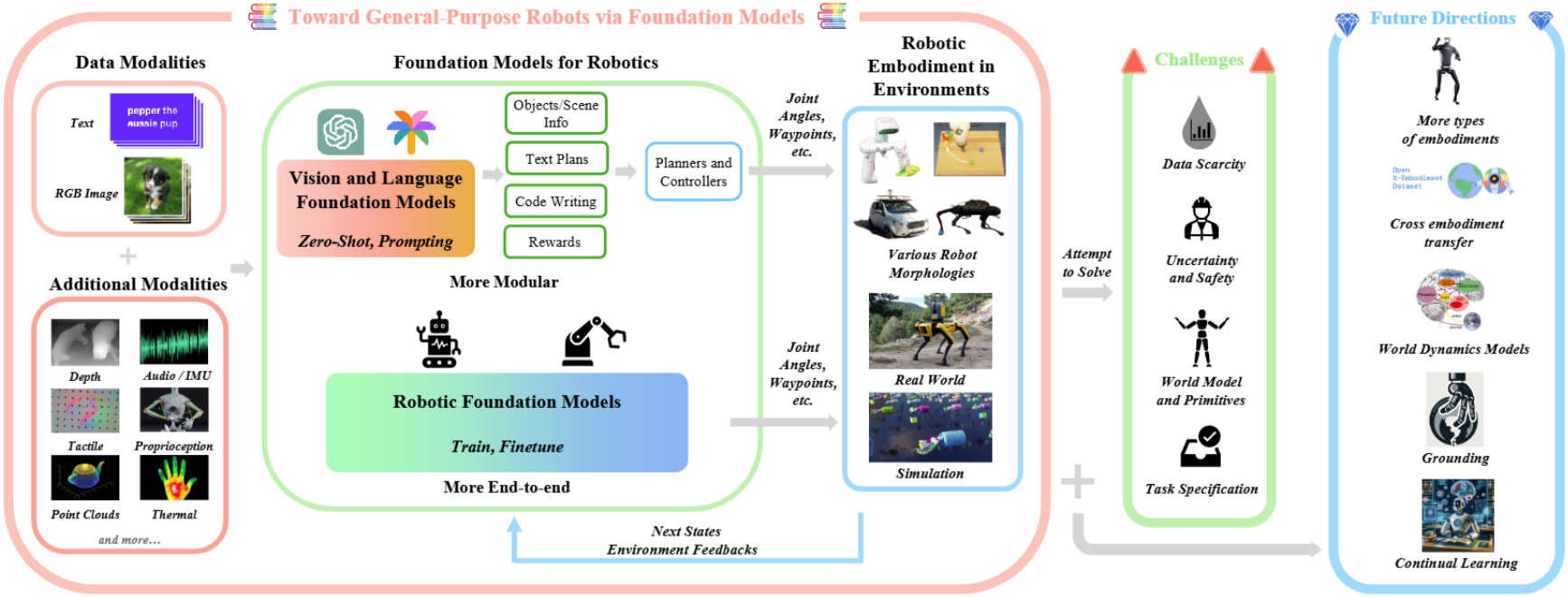
Robotic Foundation Models

- What is a Robotic Foundation Model?
  - No explicit representation of states / transition functions
  - A policy that maps (observation/state, goal) to action

- Current Foundational Vision-and-Language Models?
  - The output may not always be perfect
  - It will always generate something reasonable

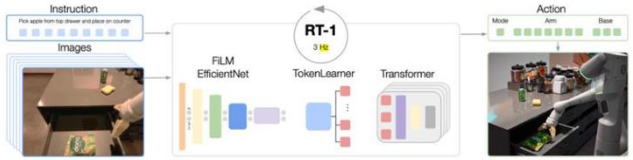
- Robotic Foundation Models
  - The synthesized action may not always be optimal.
  - The generated trajectory will always be beautiful and reasonable.

Current Research Landscape

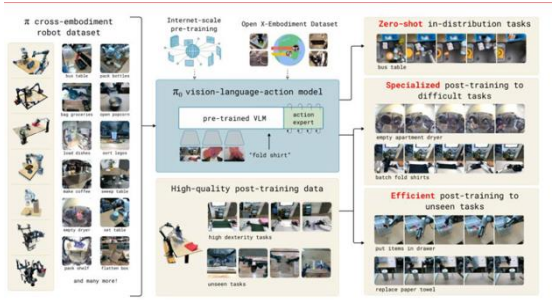


Hu*, Xie*, Jain*, et al. Toward General-Purpose Robots via Foundation Models: A Survey and Meta-Analysis, 2024

Robotic Foundation Models



RT -1 (Dec. 2022)



Pi-Zero (Oct. 2024)

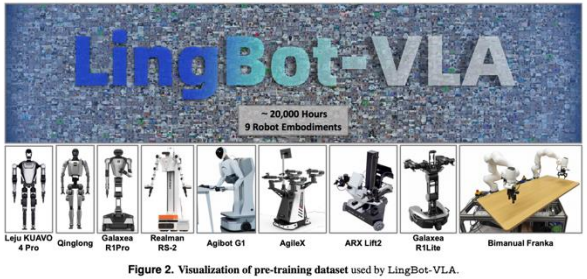
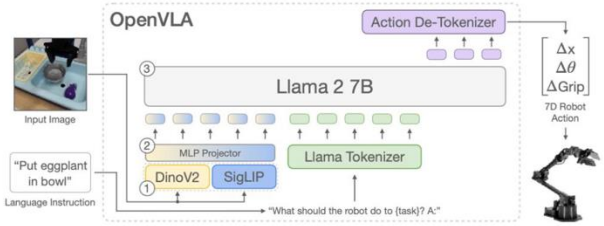


Figure 2. **Visualization of pre-training dataset** used by LingBot-VLA.

Lingbot VLA (Jan. 2026)



OpenVLA (Jun. 2024)

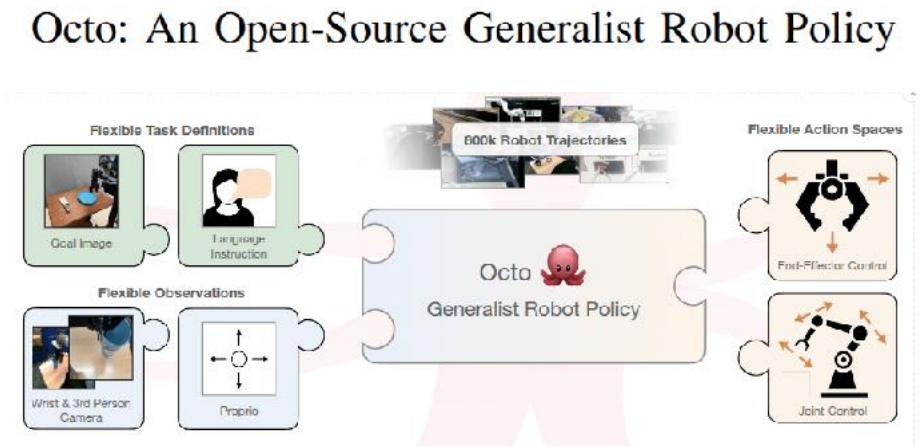
Gemini Robotics (Mar. 2025)

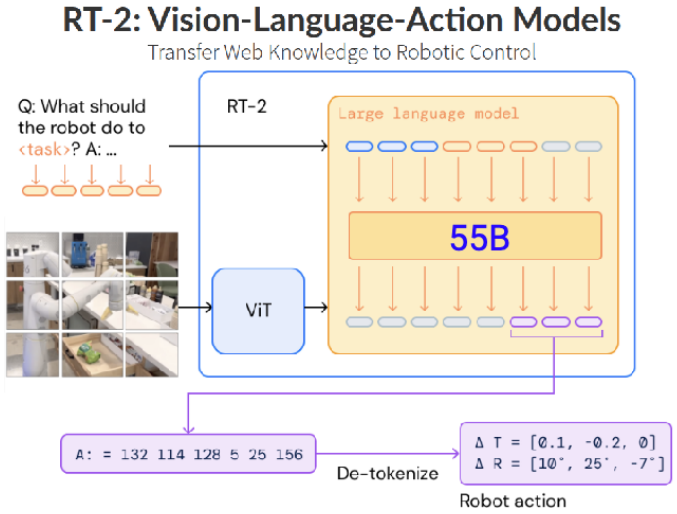
Disclaimer

Due to the rapidly changing nature and the rich body of literature in the field of foundation models for robotics, this lecture may not include all the literatures and there may be inaccuracies or mistakes in this presentation.

Existing works are:

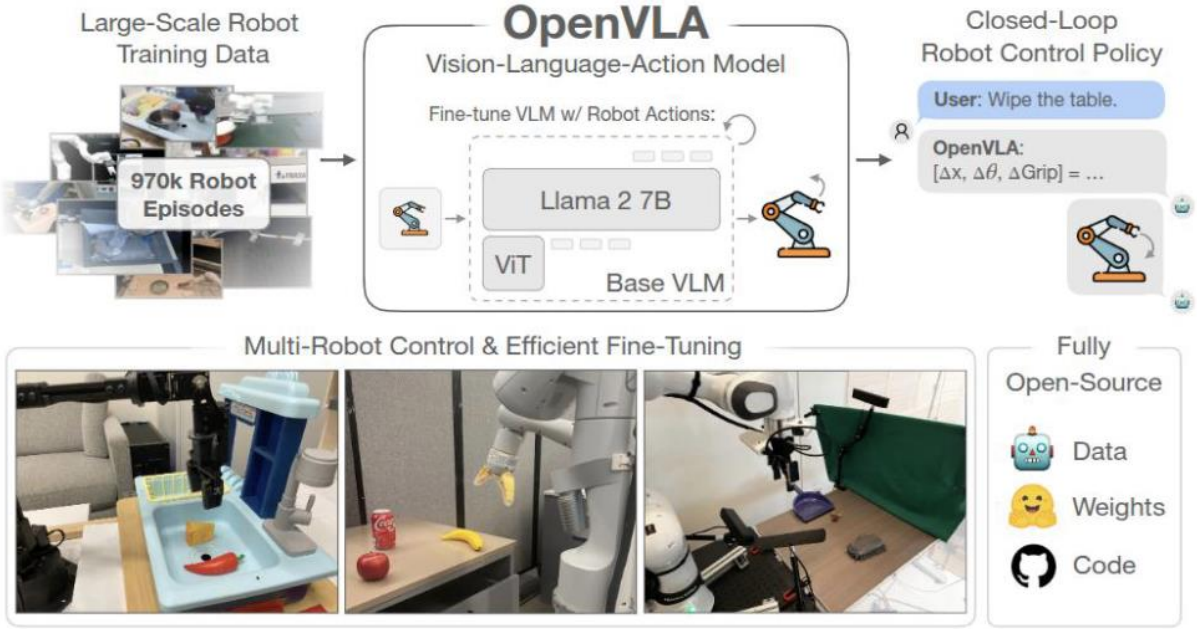
- 1. Parameter-wise Heavy (~55B parameters)
- 2. Closed-Source
- 3. Lacking fine-tuning exploration





OpenVLA l Robotic Vision - Language - Action model

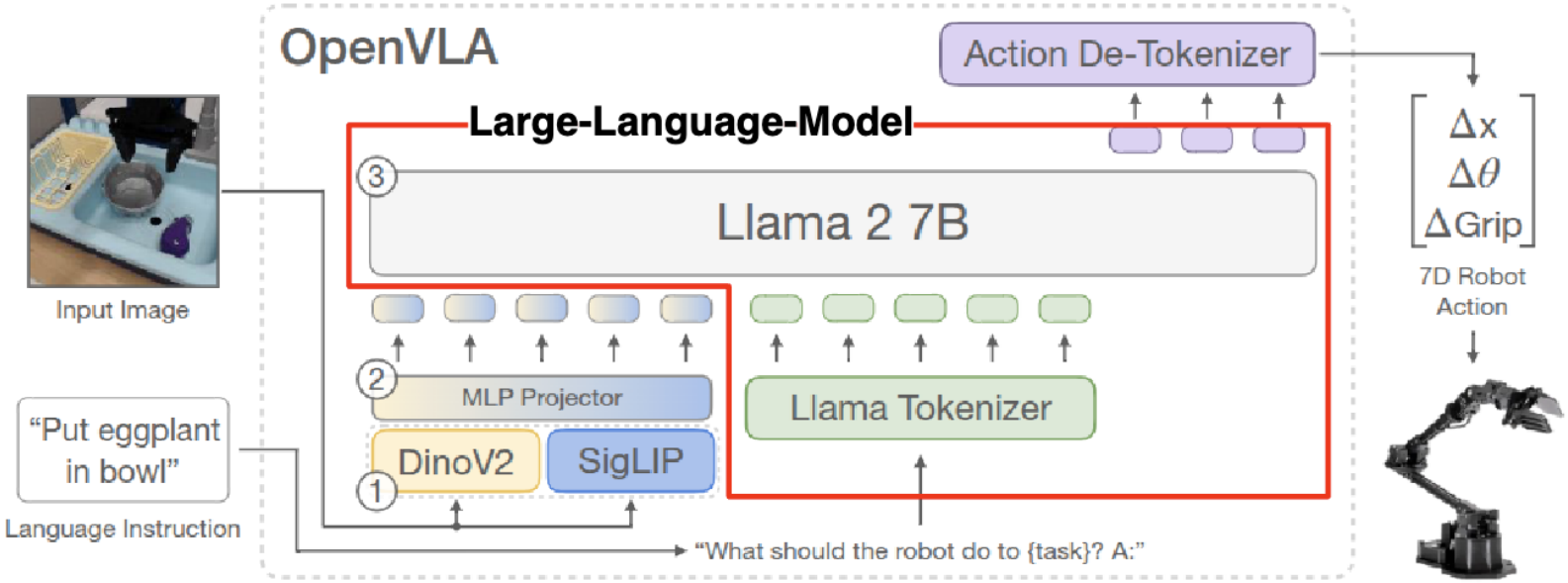
- consists of 7B parameter, fully open-source, support efficient fine-tuning



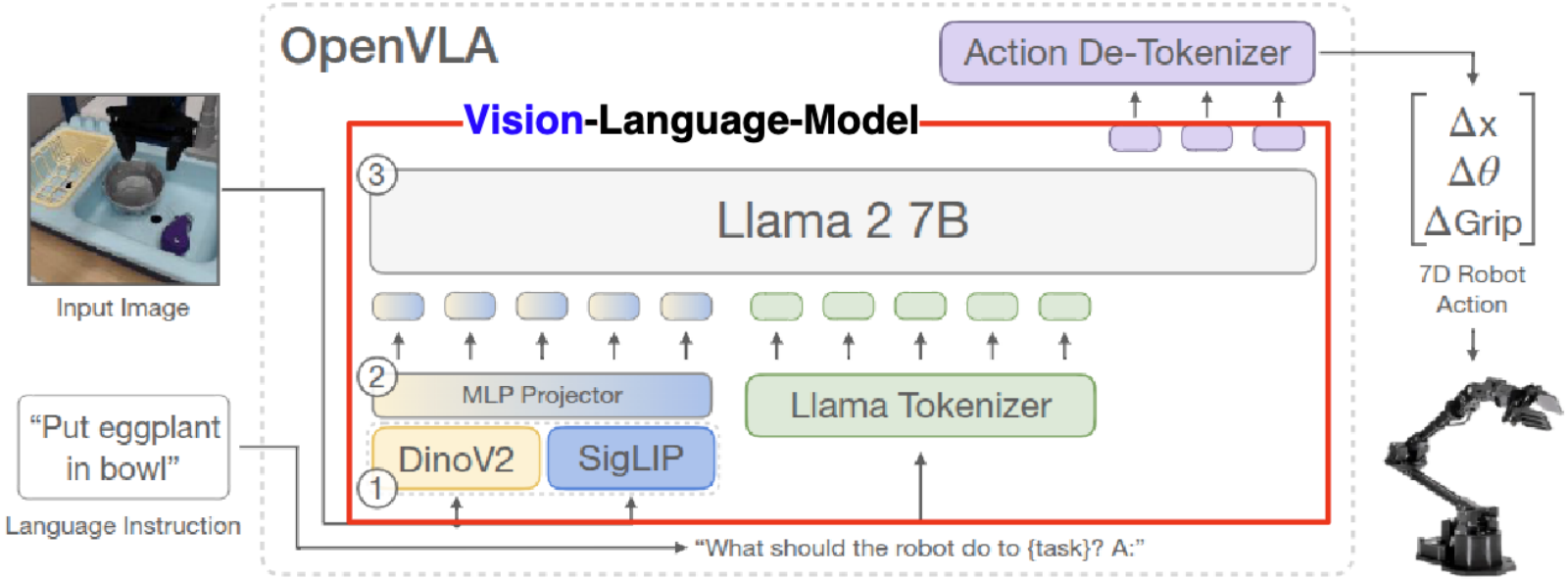
OpenVLA l Contribution

- Outperform SOTA RT-2-X (55B) by 16.5% in absolute task success rate
  - work across 29 tasks, multiple robot embodiments, with fewer parameters(7B)
- Demonstrate effectiveness of modern parameter-efficient fine-tuning and quantization
- First open-source generalist VLA thus supports future research

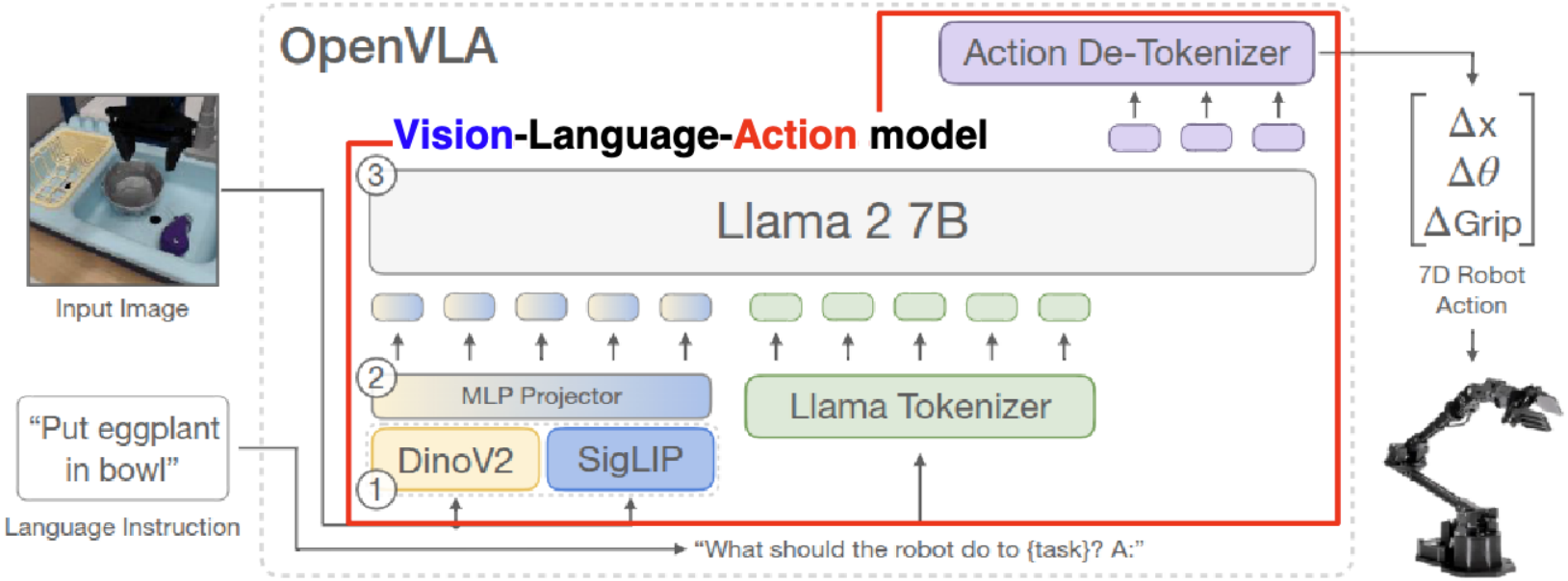
OpenVLA l Architecture



OpenVLA l Architecture



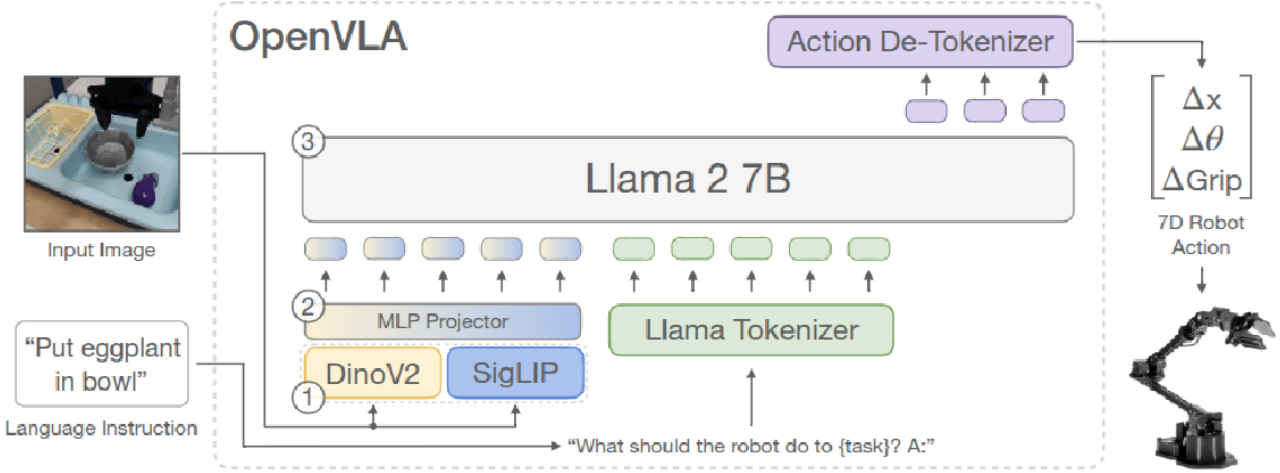
OpenVLA l Architecture



OpenVLA l Architecture

OpenVLA architecture consists of three key components

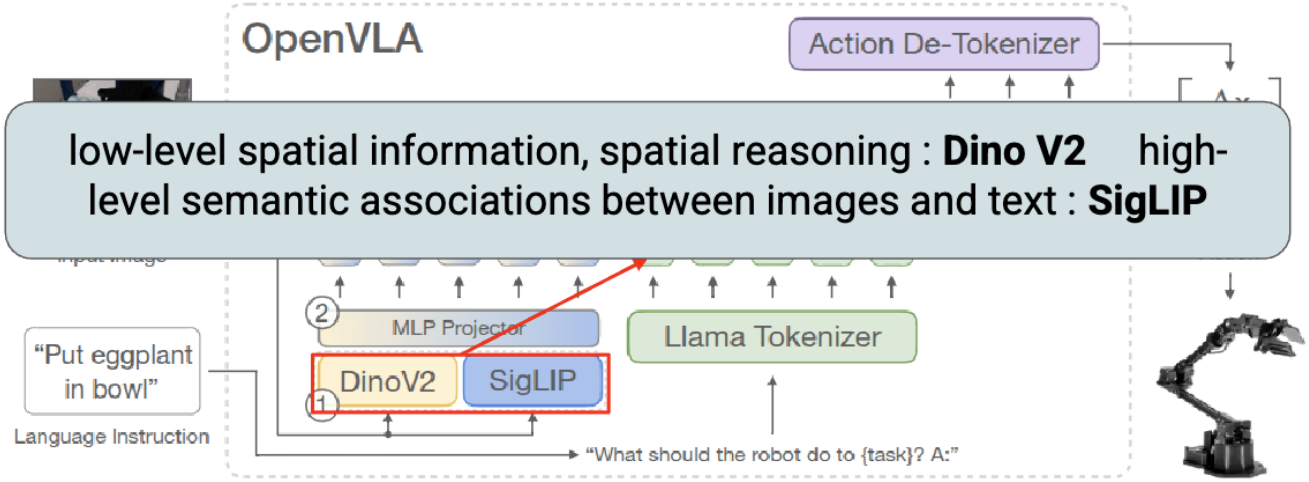
- 1. Vision encoder that concatenates Dino V2 and SigLIP features
- 2. Projector that maps visual features to the language embedding space
- 3. Llama 2 7B-parameter large language model



OpenVLA l Architecture

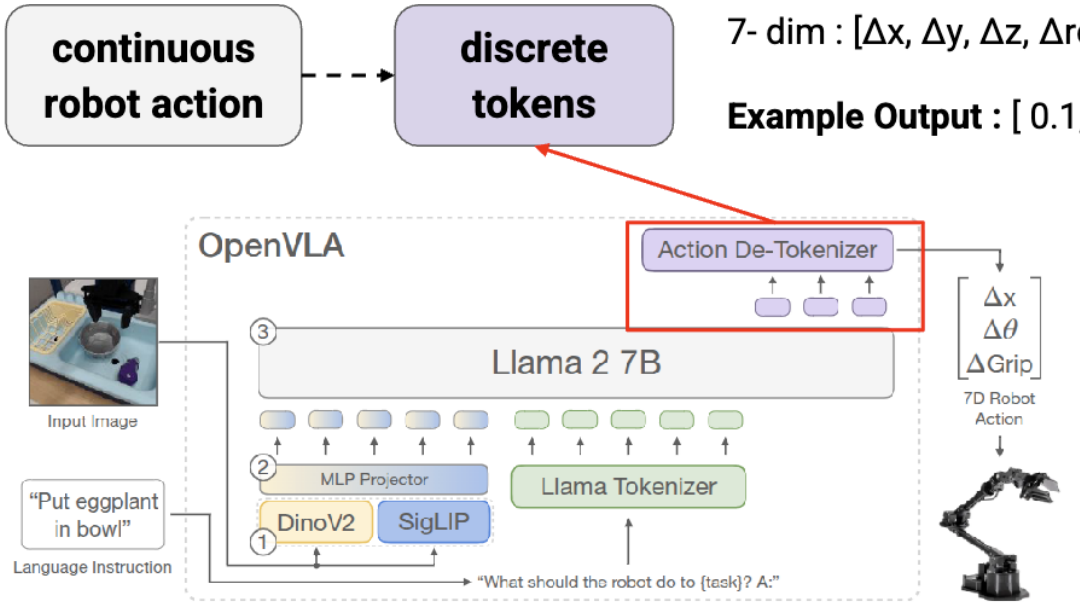
OpenVLA architecture consists of three key components

- **1. Vision encoder that concatenates Dino V2 and SigLIP features**
- 2. Projector that maps visual features to the language embedding space
- 3. Llama 2 7B-parameter large language model



OpenVLA l Architecture

Enabling VLM to predict robot actions : next-token prediction (CE loss)



OpenVLA l Training

Training Data : Open X-Embodiment dataset 70 robot dataset with 2M trajectories

Curated under below conditions, selected 970k robot episodes

1. Single arm manipulations, with 3rd person view camera
2. 2. Ensure a balanced mix of embodiments, tasks, scenes

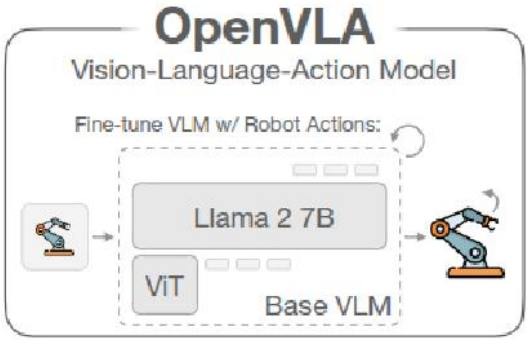
OpenVLA Training Dataset Mixture	
Fractal [92]	12.7%
Kuka [45]	12.7%
Bridge[6, 47]	13.3%
Taco Play [93, 94]	3.0%
Jaco Play [95]	0.4%
Berkeley Cable Routing [96]	0.2%
Roboturk [97]	2.3%
Viola [98]	0.9%
Berkeley Autolab UR5 [99]	1.2%
Toto [100]	2.0%
Language Table [101]	4.4%
Stanford Hydra Dataset [102]	4.4%
Austin Buds Dataset [103]	0.2%
NYU Franka Play Dataset [104]	0.8%
Furniture Bench Dataset [105]	2.4%
UCSD Kitchen Dataset [106]	<0.1%
Austin Sailor Dataset [107]	2.2%
Austin Sirius Dataset [108]	1.7%
DLR EDAN Shared Control [109]	<0.1%
IAMLab CMU Pickup Insert [110]	0.9%
UTAustin Mutex [111]	2.2%
Berkeley Fanuc Manipulation [112]	0.7%
CMU Stretch [113]	0.2%
BC-Z [55]	7.5%
FMB Dataset [114]	7.1%
DobbE [115]	1.4%
DROID [11]	10.0%[6]

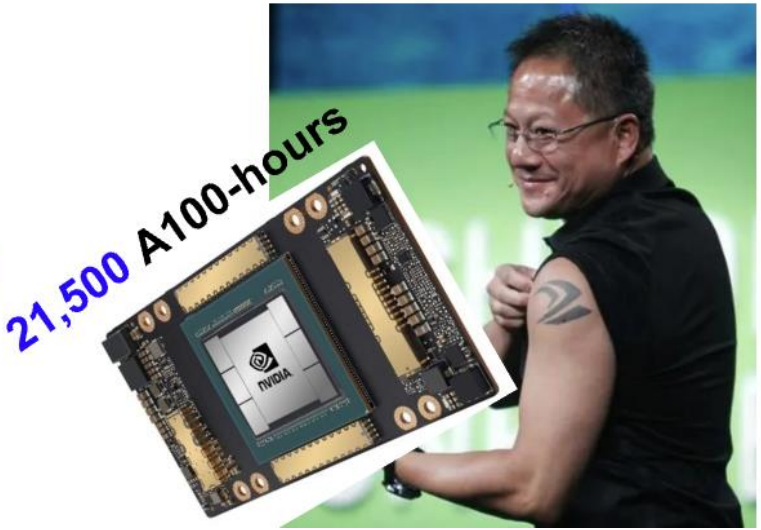
OpenVLA l Training

Other considerations during training :

OpenVLA model is trained on a cluster of 64 A100 GPUs for 14 days

- Inference : 15GB of GPU memory when loaded bfloat16





OpenVLA l Experiments

Experiments validate 3 key aspects of OpenVLA:

1. Performance as a Generalist Policy
Zero-shot generalization tests

2. Effectiveness of Fine-tuning
New robot setups & tasks

3. Performance with Limited Hardware
Parameter-efficient FT, quantization

OpenVLA l Generalization

Settings : Robot and tasks from **pre-trained data**

WidowX robot from BridgeData V2 evaluation

Mobile manipulation robot from RT-1 and RT-2 evaluation

Mobile manipulation robot

Categorizing the term "Generalization" :

Visual : unseen backgrounds, distractor objects, appearances of objects

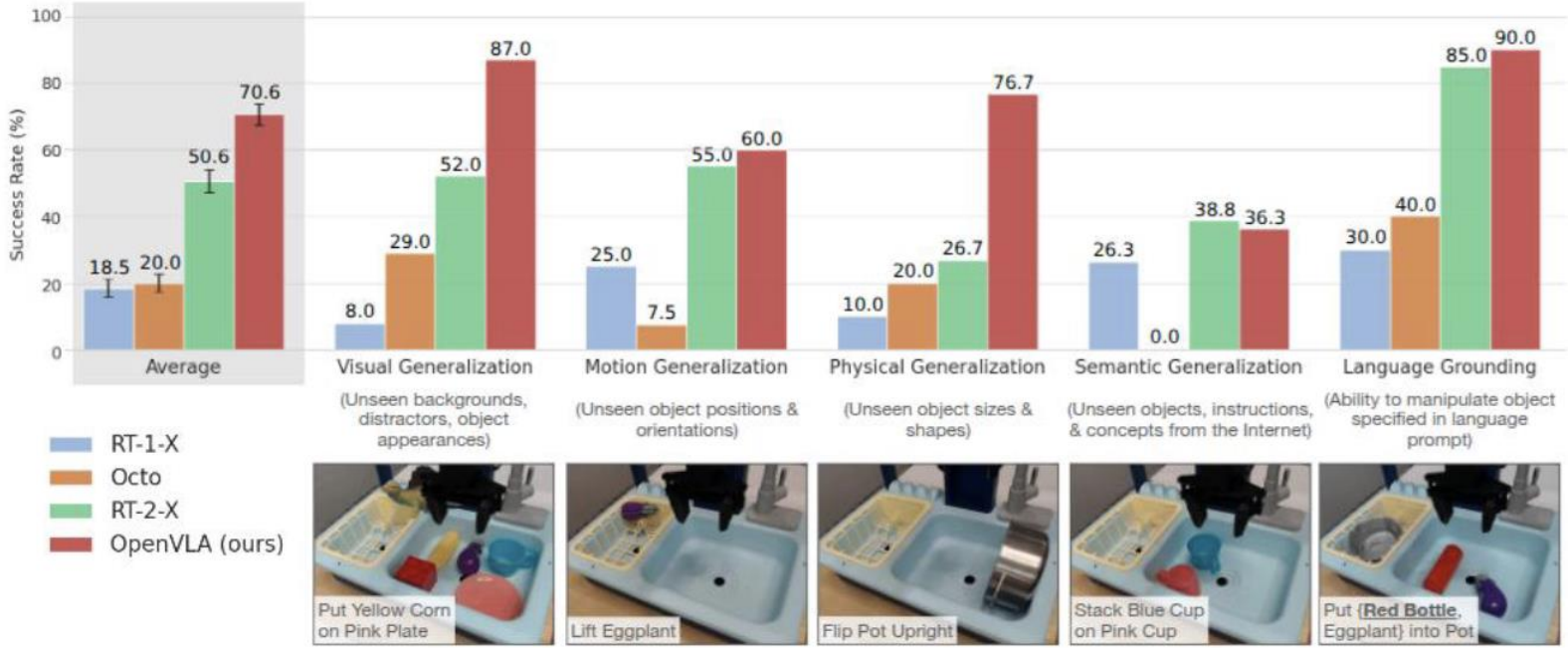
Motion : unseen object positions/orientations

Physical : unseen object sizes/shapes

Semantic : unseen target objects, instructions, concepts from the Internet

WidowX robot

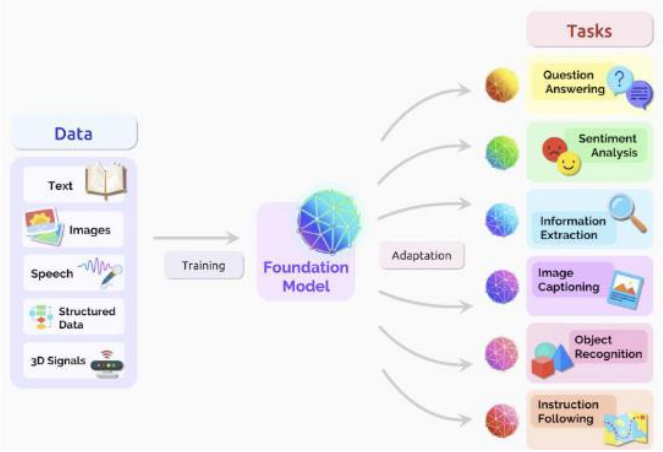
OpenVLA l Generalization

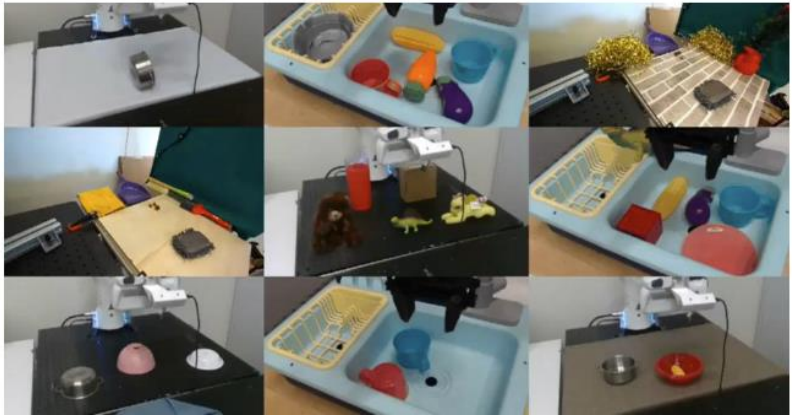


OpenVLA l Efficient Fine-Tuning

"Adapting a pretrained VLA model to a **new robot, new environment, or new task**"

- Quick adaptation to a new setup, with a much smaller dataset
- Implicitly captures per-setup differences
  - Camera pose, embodiment, environment, reference frame, ...

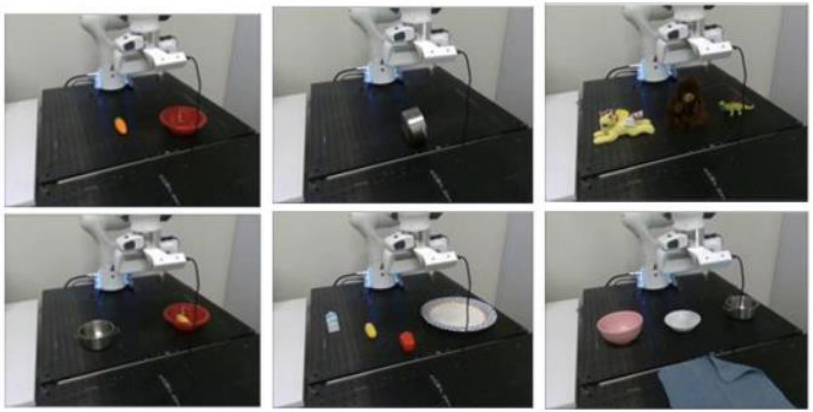




OpenVLA | Efficient Fine-Tuning

"Is OpenVLA adaptable to new robot setups and tasks?"

- Prepared Franka robot arm setups **not in pretraining data**
- Models trained / fine-tuned on 7 tasks, 10-150 demonstrations each
  - Diffusion policies, Octo, OpenVLA (scratch and pretrained)



OpenVLA l Efficient Fine-Tuning

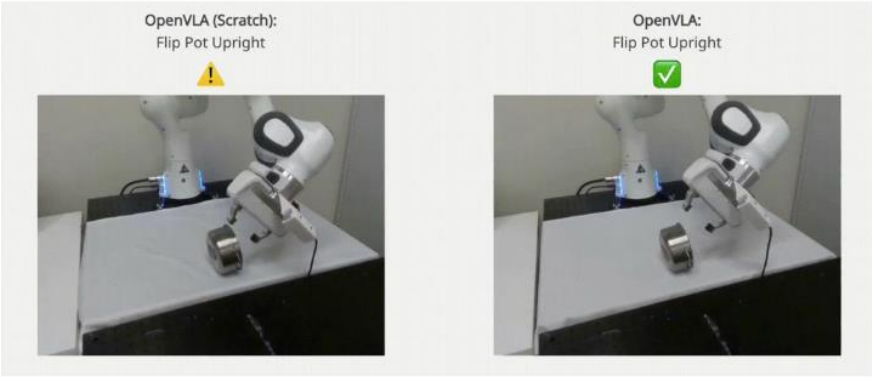
1. Diffusion Policy → good for narrow, single-instruction tasks

OpenVLA l Efficient Fine-Tuning

1. Diffusion Policy → good for narrow, single-instruction tasks
2. Fine-tuned VLAs → better with multiple objects & language conditioning

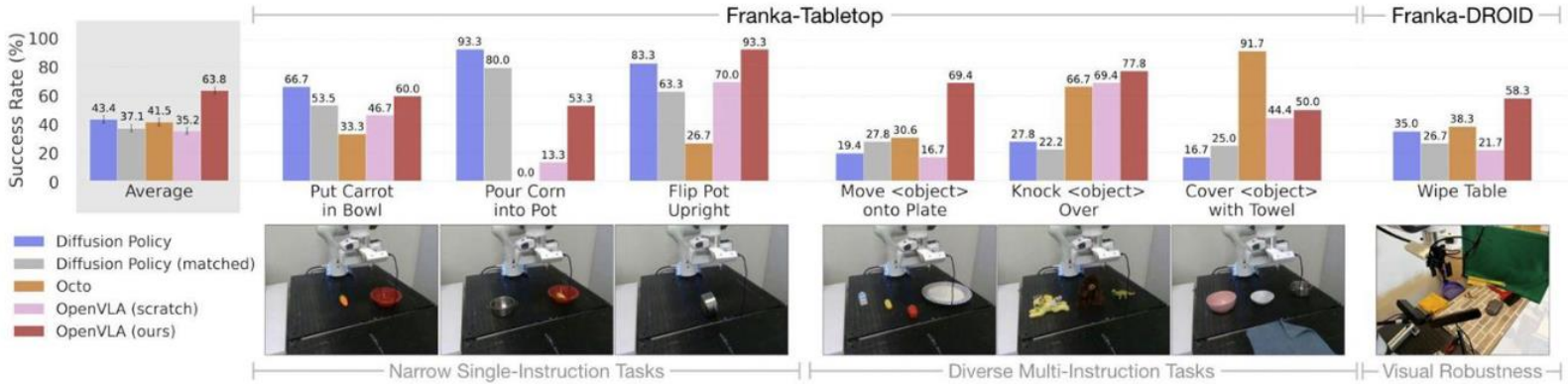
OpenVLA l Efficient Fine-Tuning

1. Diffusion Policy → good for narrow, single-instruction tasks
2. Fine-tuned VLAs → better with multiple objects & language conditioning
3. Robot data pretraining → significant performance boost



OpenVLA l Efficient Fine-Tuning

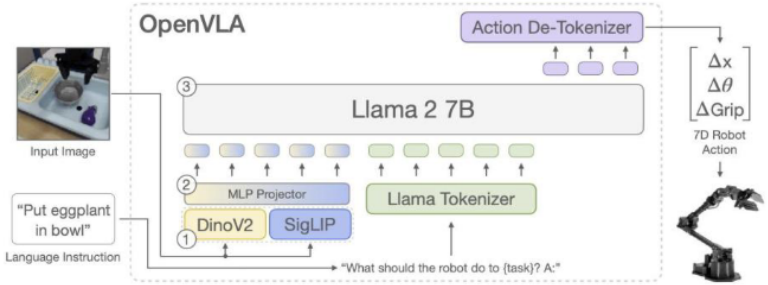
1. Diffusion Policy → good for narrow, single-instruction tasks
2. Fine-tuned VLAs → better with multiple objects & language conditioning
3. Robot data pretraining → significant performance boost
4. OpenVLA shows **strong performance across task types**

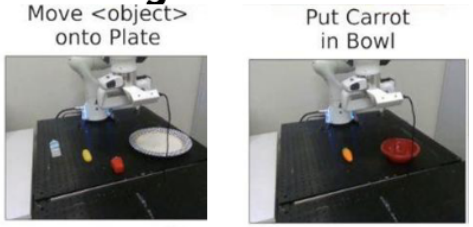


OpenVLA | Parameter-efficient FT

Tested different fine-tuning strategies

- Strategies: Full FT, Last layer only, Frozen vision, Sandwich, LoRA
- Criteria: memory requirement, performance
- Remark: LoRA is **efficient**, with **minimal performance degradation**!



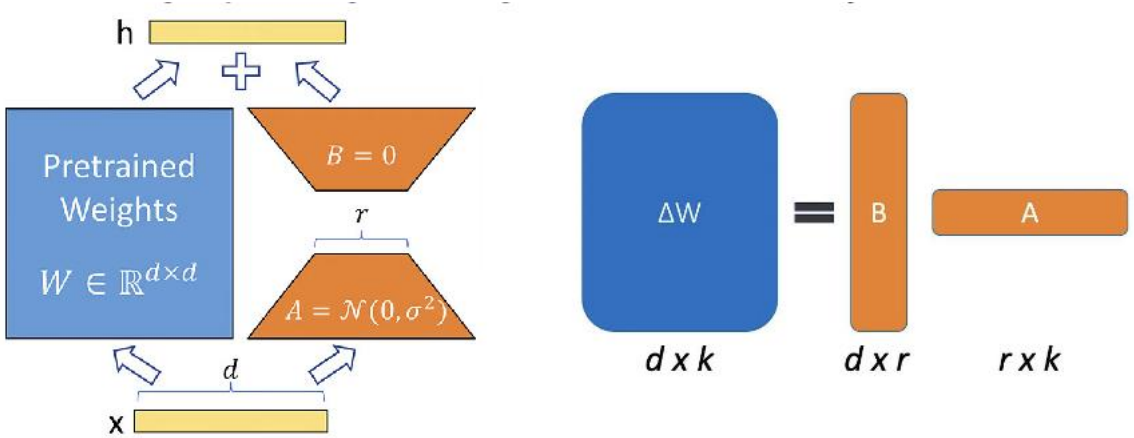


Strategy	Success Rate	Train Params ($\times 10^6$)	VRAM (batch 16)
Full FT	**69.7 ± 7.2 %**	7,188.1	163.3 GB*
Last layer only	30.3 ± 6.1 %	465.1	51.4 GB
Frozen vision	47.0 ± 6.9 %	6,760.4	156.2 GB*
Sandwich	62.1 ± 7.9 %	914.2	64.0 GB
LoRA, rank=32	**68.2 ± 7.5%**	**97.6**	**59.7 GB**
rank=64	**68.2 ± 7.8%**	195.2	60.5 GB

OpenVLA | Parameter-efficient FT

LoRA: Low-Rank Adaptation of Large Language Models

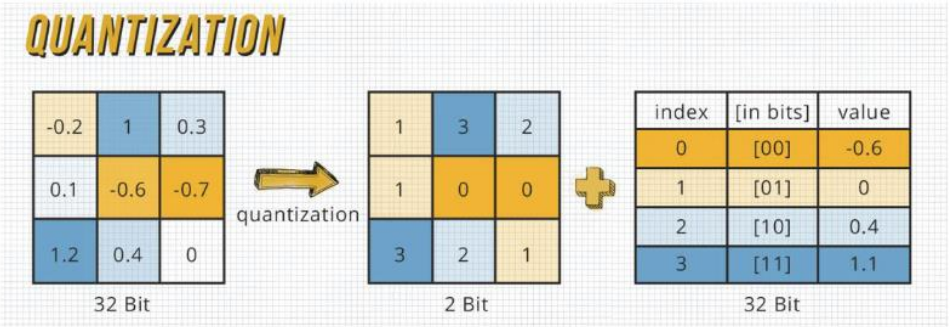
- Vanilla fine-tuning: update (d x h) weights, expensive
- LoRA: only train B and A, where BA = ΔW (weight updates)
- Insight
  - Pretrained W is near a good solution
  - For fine-tuning, updating all weights is unnecessary!



OpenVLA l Quantization

Many foundation models require large GPU memory (even **for inference**)

- Idea: reduce the precision (# of bits used) of model weights
  - Less memory footprint
  - Extra quantization / dequantization overhead
  - Increased arithmetic error
- **Memory ↔ Performance tradeoff**



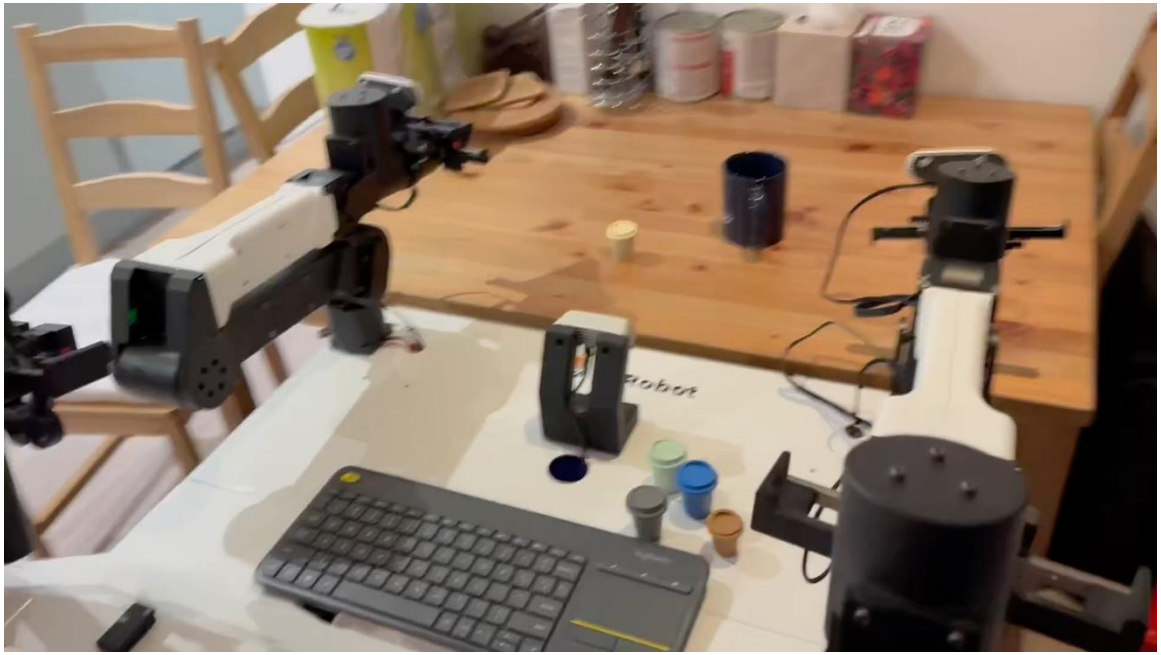
OpenVLA l Limitation

- Only support single-image observation
- Inference throughput (Hz)
- Room for performance improvement (90+ %)

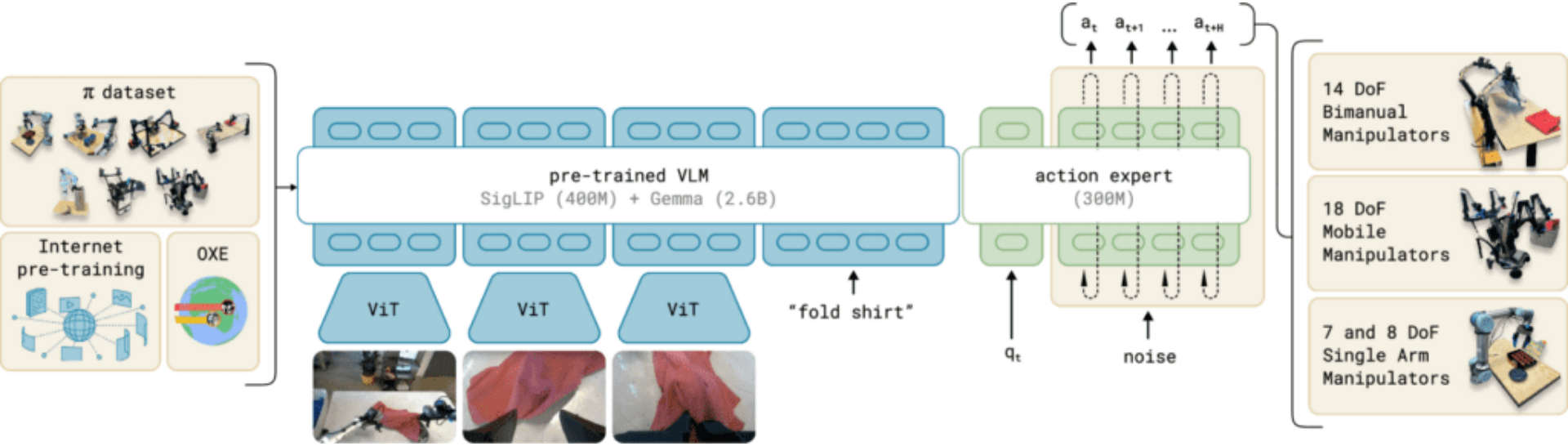
OpenVLA l Summary

- Open-source VLA model for manipulators
  - Image + language instruction → robot action
- CV & NLP advancements on robotic applications
- Towards large, ‘generalist’, widely-deployable robot models
  - Robot data pretraining → general performance
  - Fine-tuning (LoRA) → task-specific adaptation
  - Quantization → memory-efficient inference

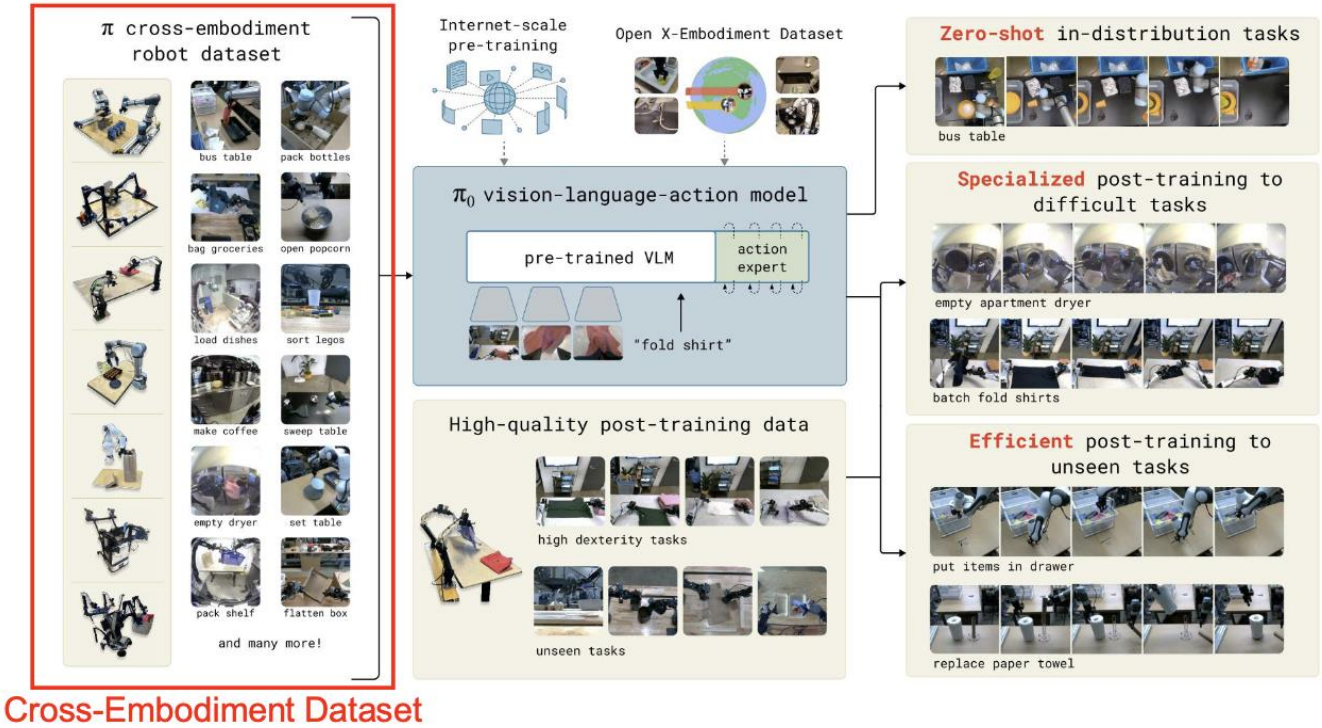
Pi-Zero by Physical Intelligence



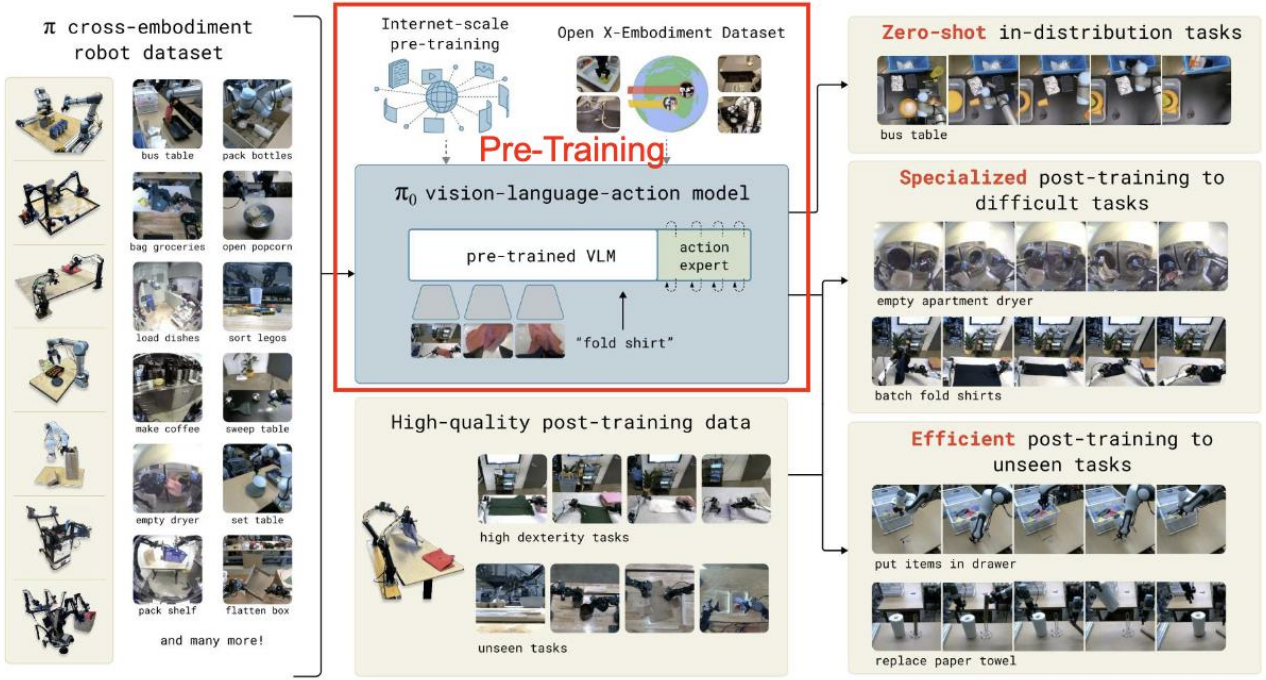
Pi-Zero by Physical Intelligence



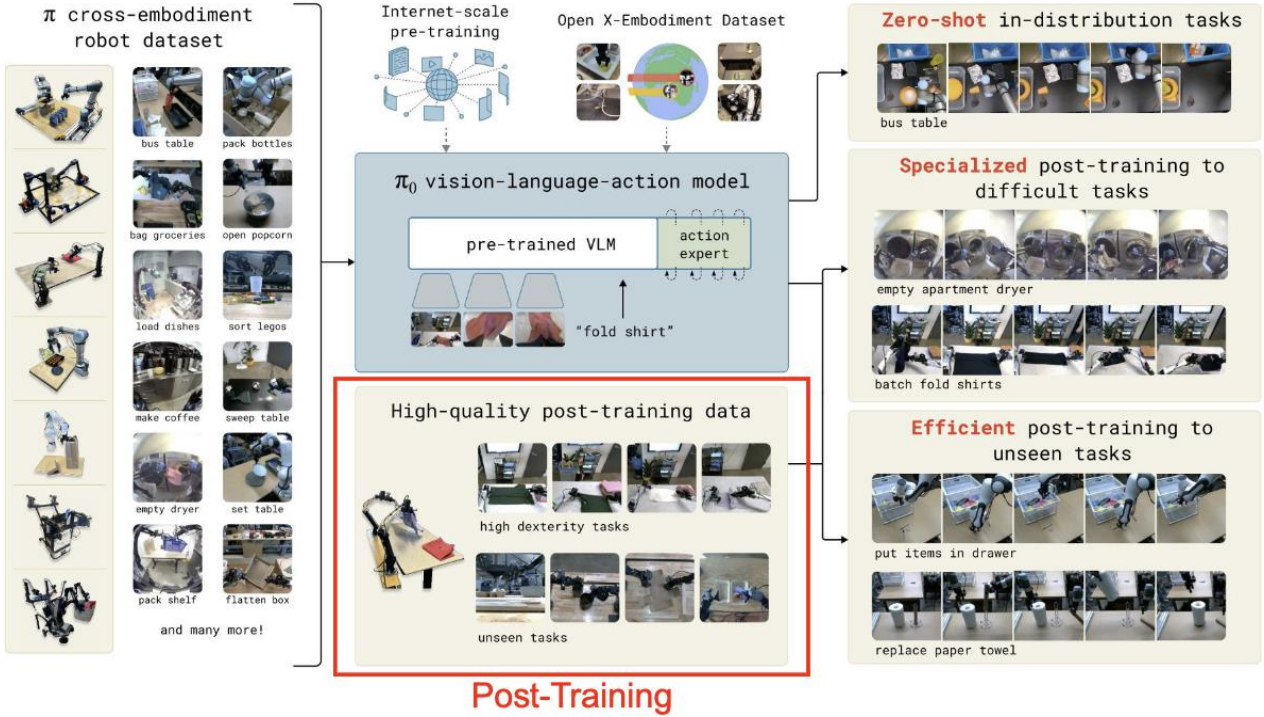
Pi-Zero by Physical Intelligence



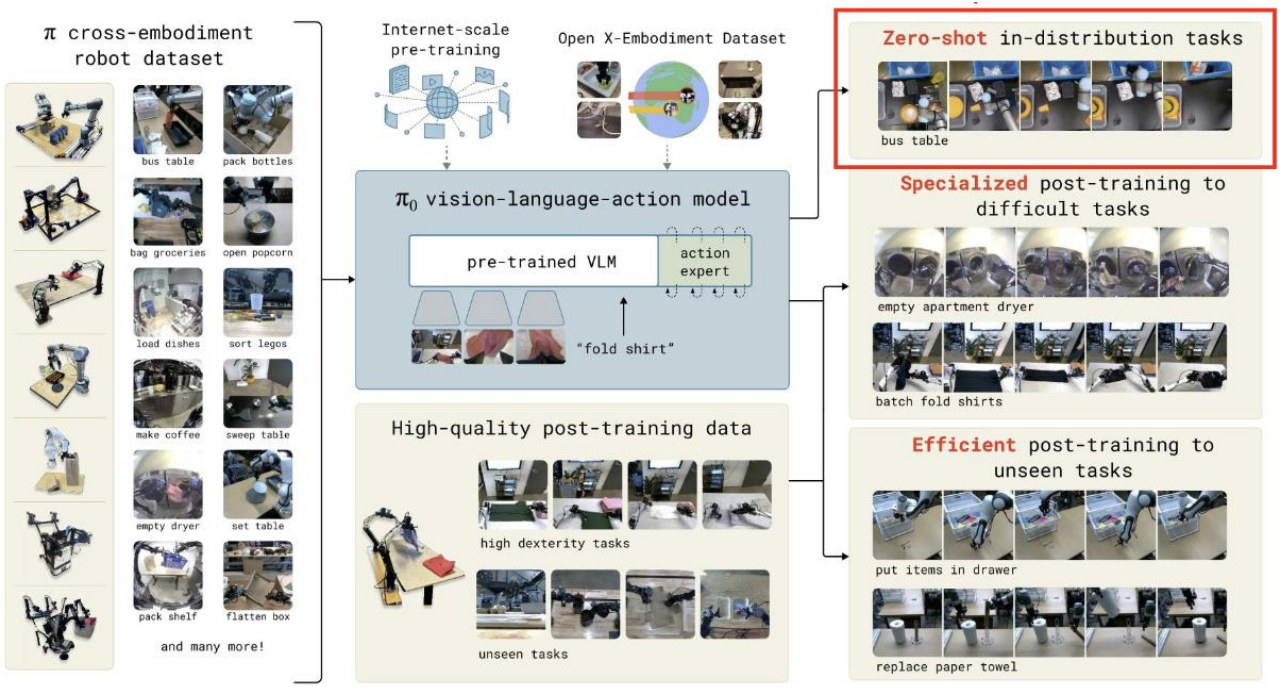
Pi-Zero by Physical Intelligence



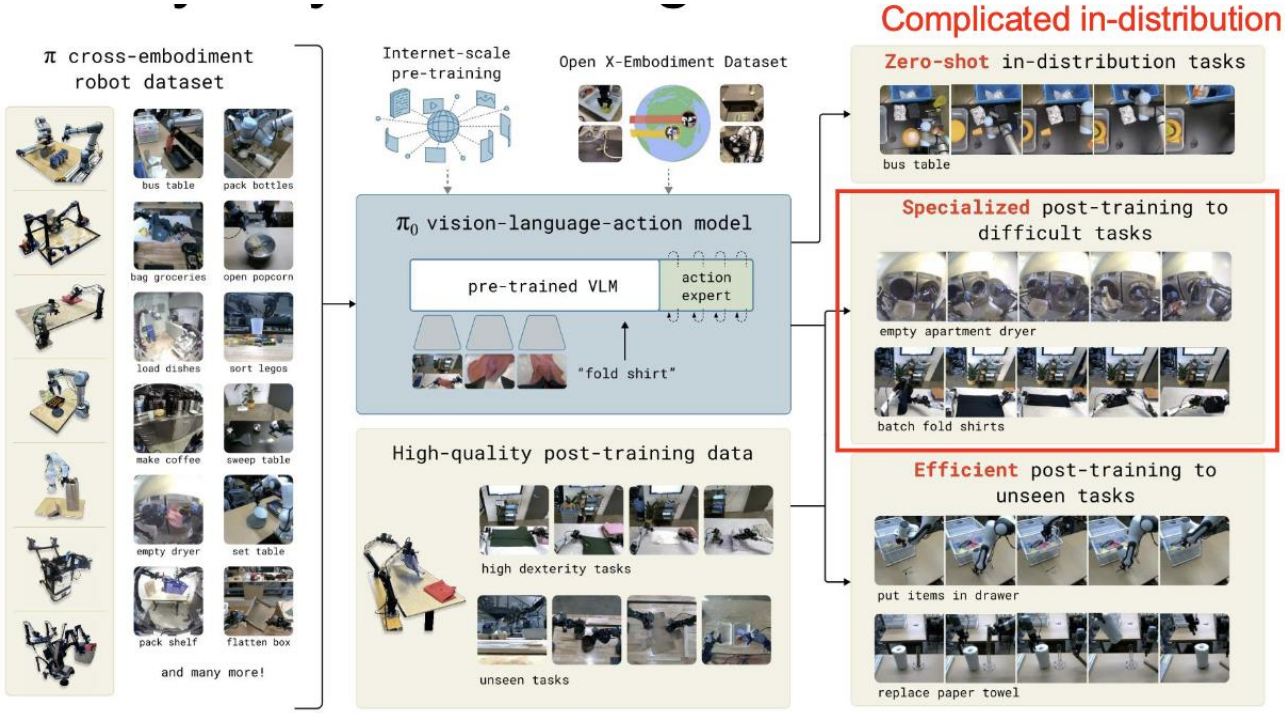
Pi-Zero by Physical Intelligence



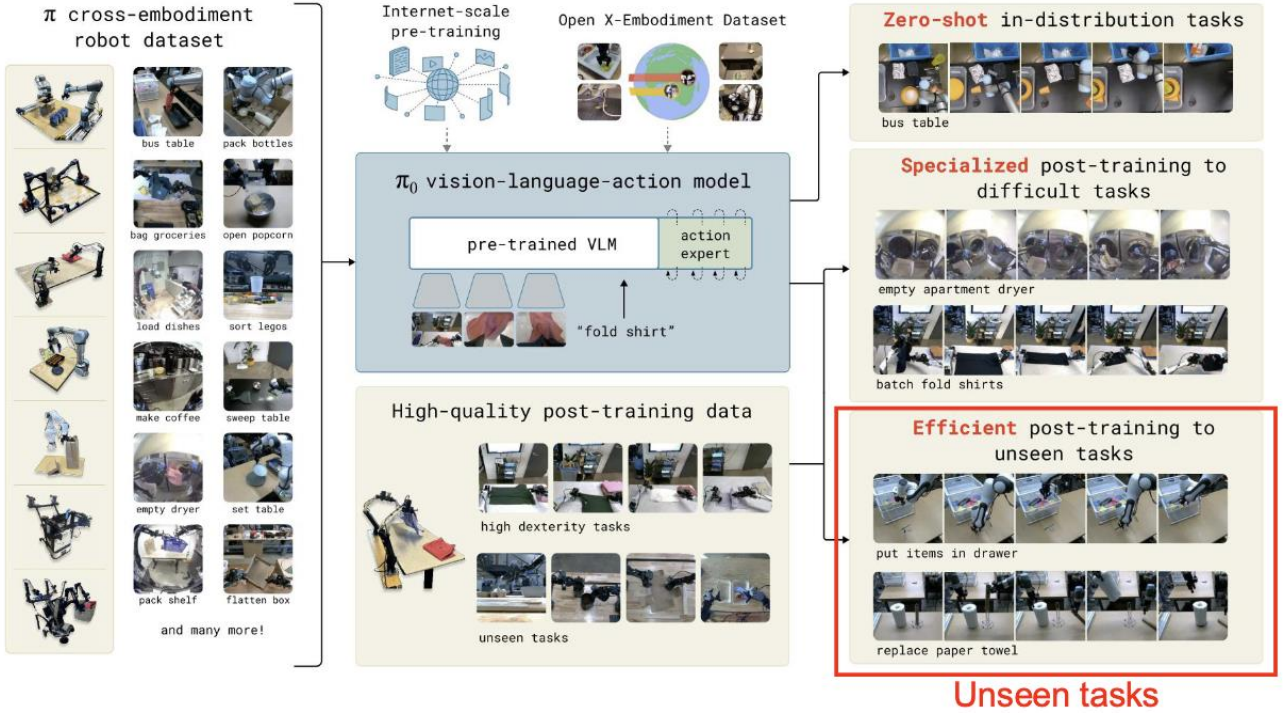
Pi-Zero by Physical Intelligence



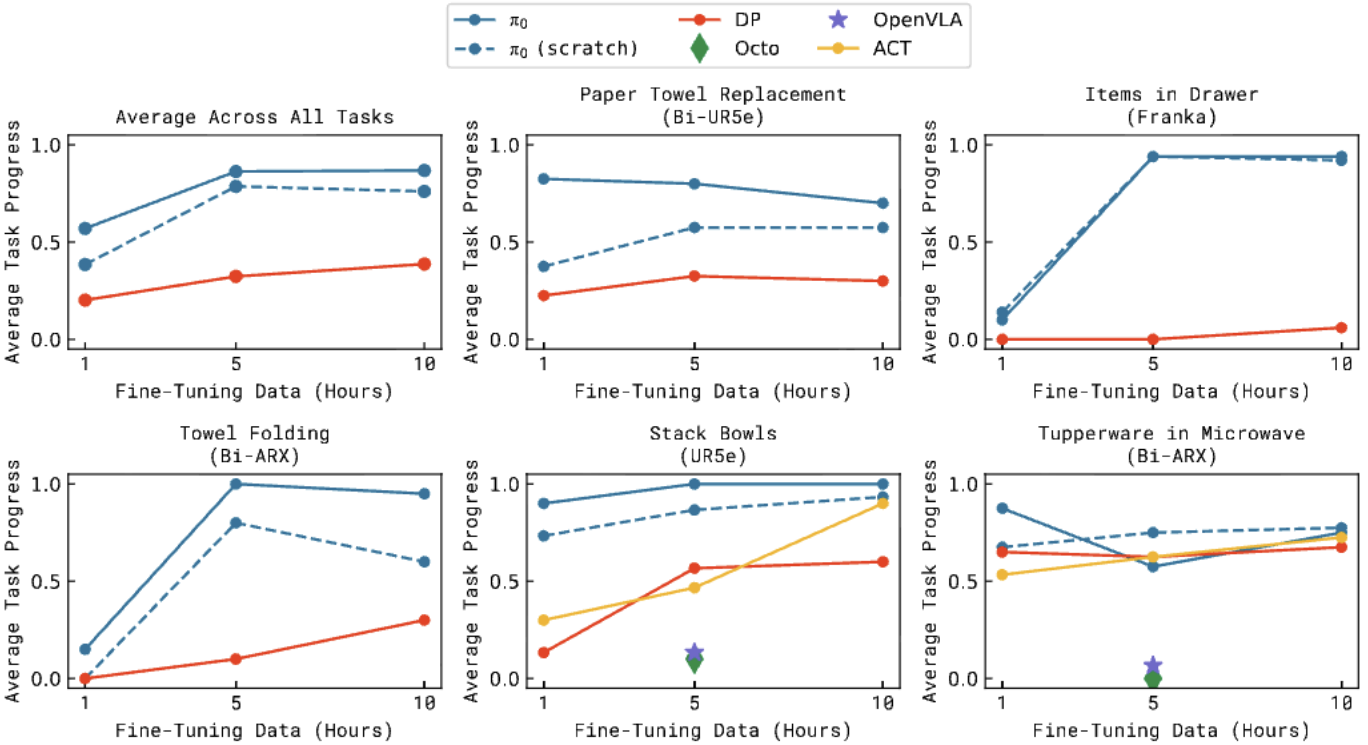
Pi-Zero by Physical Intelligence



Pi-Zero by Physical Intelligence



Pi-Zero | Performance vs OpenVLA



Pi-Zero | Performance vs OpenVLA

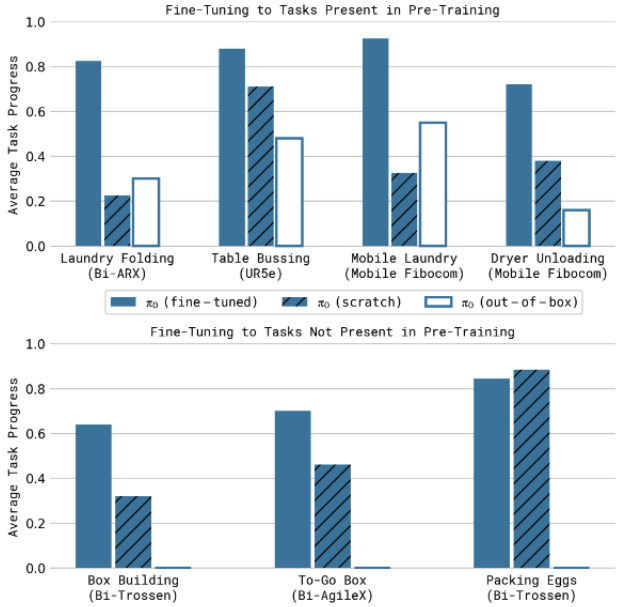


Fig. 13: **Post-training results on complex tasks** in terms of average scores over 10 trials. The full pre-trained π_0 model attains more than 50% of the maximum score across all of the tasks, and typically outperforms the ablations, with especially significant improvements on the hardest tasks.

model part	inference time
image encoders	14 ms
observation forward pass	32 ms
x10 action forward pass (flow)	27 ms
network latency (if off-board)	13 ms
total on-board inference	73 ms
total off-board inference	86 ms

TABLE I: Inference time of our model on an NVIDIA GeForce RTX 4090 GPU.

Pi-Zero, it’s the opensource community

Physical Intelligence (π)

Open Sourcing π_0

Published February 4, 2025
Email research@physicalintelligence.company
Repo Physical-Intelligence/openpi

README Apache-2.0 license

openpi

openpi holds open-source models and packages for robotics, published by the Physical Intelligence team.

Currently, this repo contains two types of models:

- the π_0 model, a flow-based diffusion vision-language-action model (VLA)
- the π_0-FAST model, an autoregressive VLA, based on the FAST action tokenizer.

For both models, we provide *base model* checkpoints, pre-trained on 10k+ hours of robot data, and examples for using them out of the box or fine-tuning them to your own datasets.

This is an experiment: π_0 was developed for our own robots, which differ from the widely used platforms such as ALOHA and DROID, and though we are optimistic that researchers and practitioners will be able to run creative new experiments adapting π_0 to their own platforms, we do not expect every such attempt to be successful. All this is to say: π_0 may or may not work for you, but you are welcome to try it and see!

Other Definition of VLMs

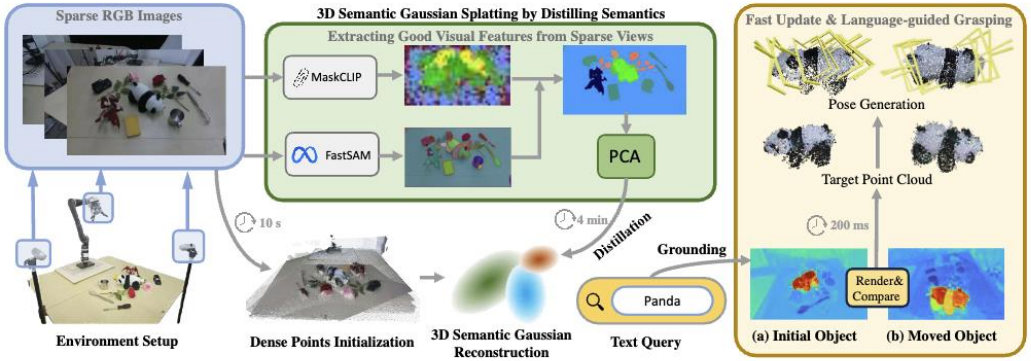


Fig. 2: Our architecture. It starts with collecting sparse view images and generating dense point clouds to initialize 3DGS. Next, we integrate FastSAM and MaskCLIP to generate average features within each mask. Then, PCA is applied to compress the whole average features in a low dimension, then distilled into 3DGS. Given an open-vocabulary language instruction, our system can locate the target object and generate appropriate grasp poses. When scene changes, the Render-and-Compare strategy enables fast scene updates.

Fig. 3: Comparison of dense point initialization v.s. sparse point initialization in sparse view images. Initializing with sparse points often leads to overfitting with sparse view images.

SparseGrasp: Robotic Grasping via 3D Semantic Gaussian Splatting from Sparse Multi-View RGB Images